

Faultline Execution Plan

Every task from empty repository to application-ready system: the master flowchart, then each task's what, why, how, and deliverable. Companion to the Faultline specification; no code here — this is the map you build from.

DATE 2026-08-21 PHASES 8 · TASKS 58 · GATES 8 TIMELINE 16–19 WEEKS PART-TIME
REV 9 · POST-REVIEW-7

CONTENTS

- 1. Phase overview and how to read this plan
- 2. Master flowchart
- 3. Phase 0 — Foundations
- 4. Phase 1 — The World (target environment)
- 5. Phase 2 — Platform core
- 6. Phase 3 — Investigation agents
- 7. Phase 4 — Evaluation harness (parallel track)
- 8. Phase 5 — Surfaces & MVP ship
- 9. Phase 6 — Production hardening
- 10. Phase 7 — Evidence & launch
- 11. Master deliverables checklist
- 12. Working rules

§1 Phase overview

The build is eight phases. Phases run mostly in sequence, with one deliberate exception: the **evaluation harness (Phase 4) starts in week 3 and runs as a parallel track** — every agent decision from Phase 3 onward is made against eval numbers, never against vibes. Each phase ends at a **gate**: a concrete, demonstrable condition. You do not move on until the gate passes; a gate that passes from a clean `git clone` is the definition of done.

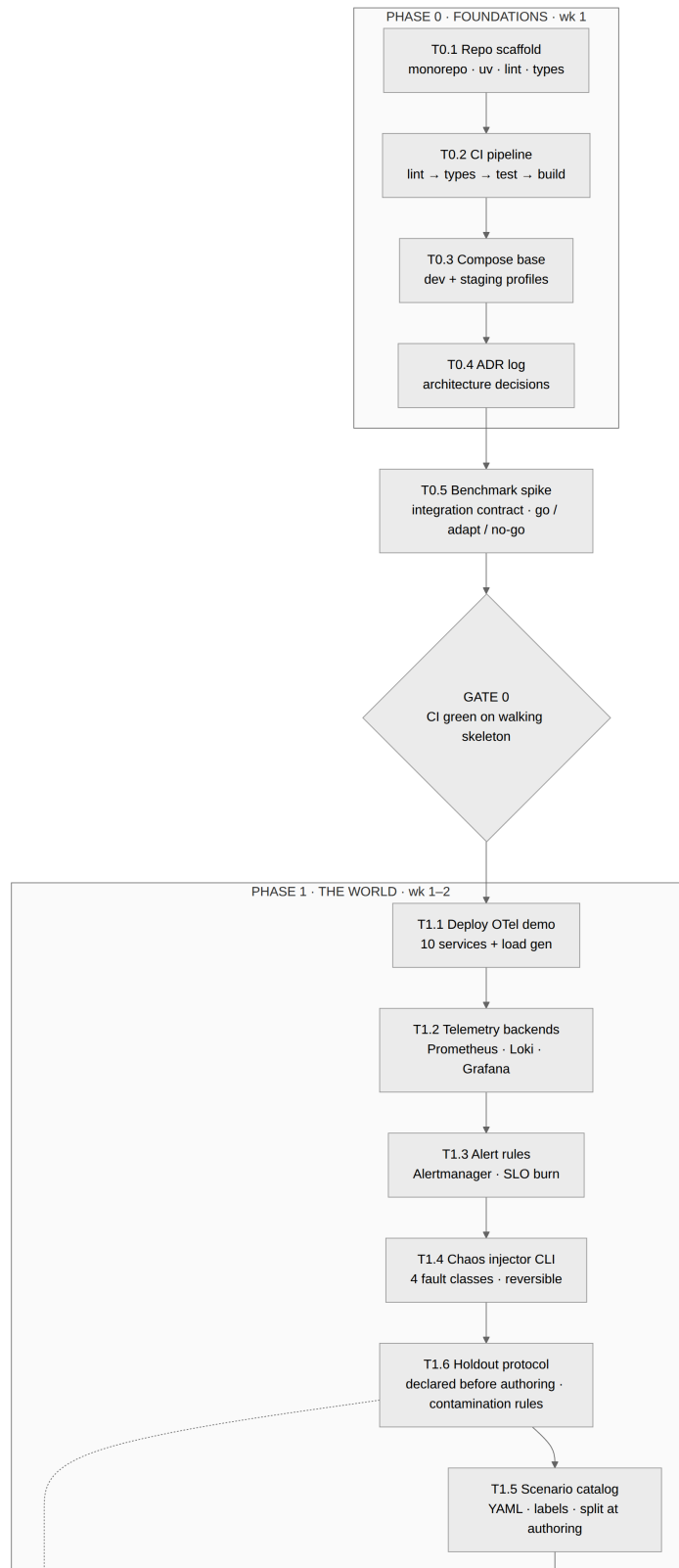
Phase	Name	Weeks	Goal in one line	Gate (exit condition)
P0	Foundations	1	Repo, CI, and tooling discipline before any feature	G0 · CI green on an empty walking skeleton
P1	The World	1–2	A real, instrumented microservices environment that can break on command	G1 · injected fault → alert fires → visible on dashboards, zero AI involved

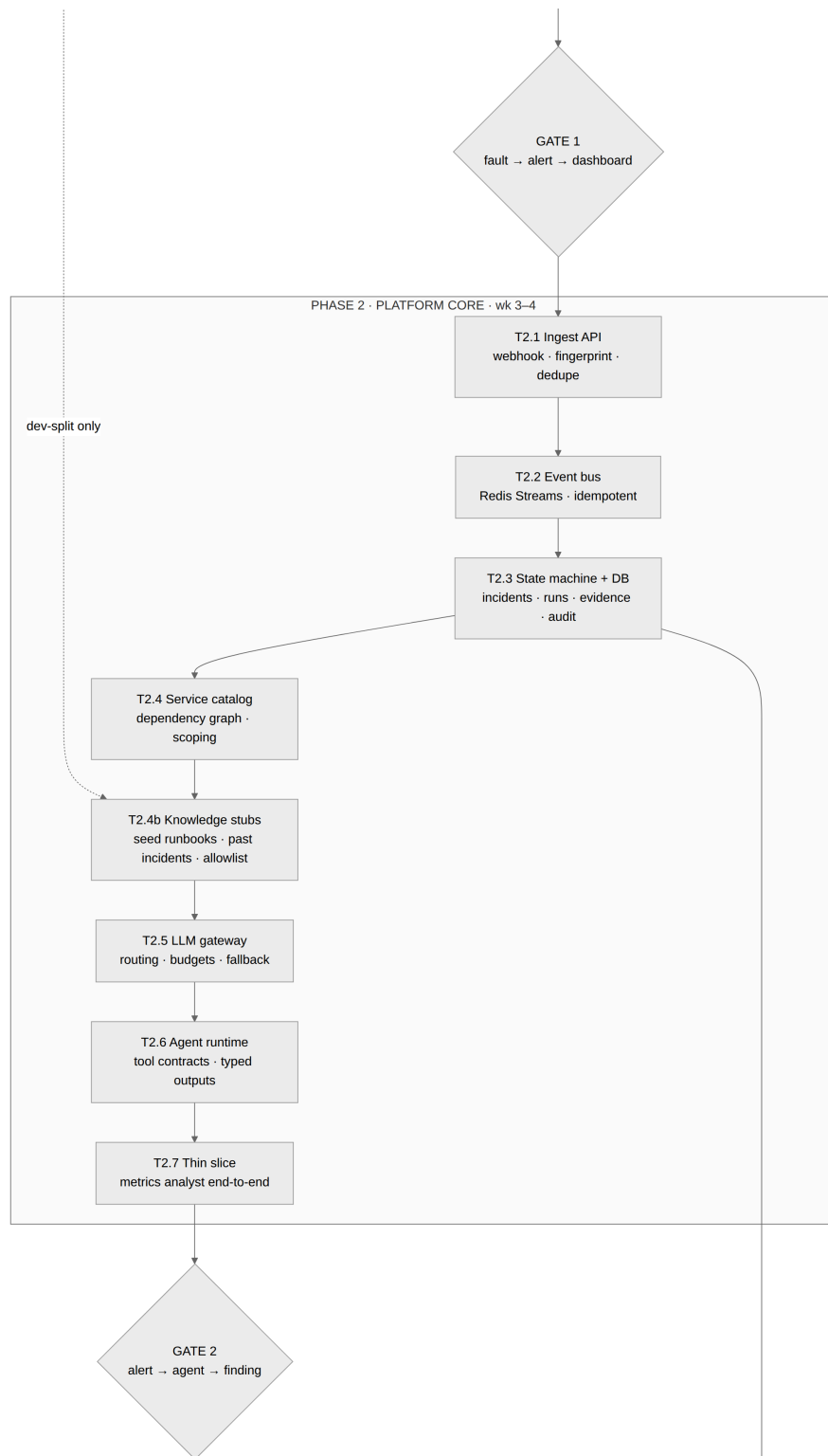
Phase	Name	Weeks	Goal in one line	Gate (exit condition)
P2	Platform core	3–4	Alert-to-incident plumbing, state machine, agent runtime	G2 · one alert → one agent → one persisted, rendered finding
P3	Investigation agents	5–7	The full multi-agent investigation pipeline with cited RCA	G3 · end-to-end investigation passes on 3 scenario classes
P4	Evaluation harness	3–7	Ground-truth scoring loop over the scenario catalog	G4 · one command runs and scores all 10 scenarios into a report
P5	Surfaces & MVP	8–9	UI, docs, video — a shippable, talkable MVP	G5 · full demo runs from clean clone; MVP tagged
P6	Production hardening	10–13	Action plane, RAG, traces, self-observability, reliability	G6 · approval-gated remediation works; injection + storm tests pass
P7	Evidence & launch	13–19	Benchmarks, ablations, dashboard, public launch pack	G7 · repo + video + benchmark and ablation reports are application-ready

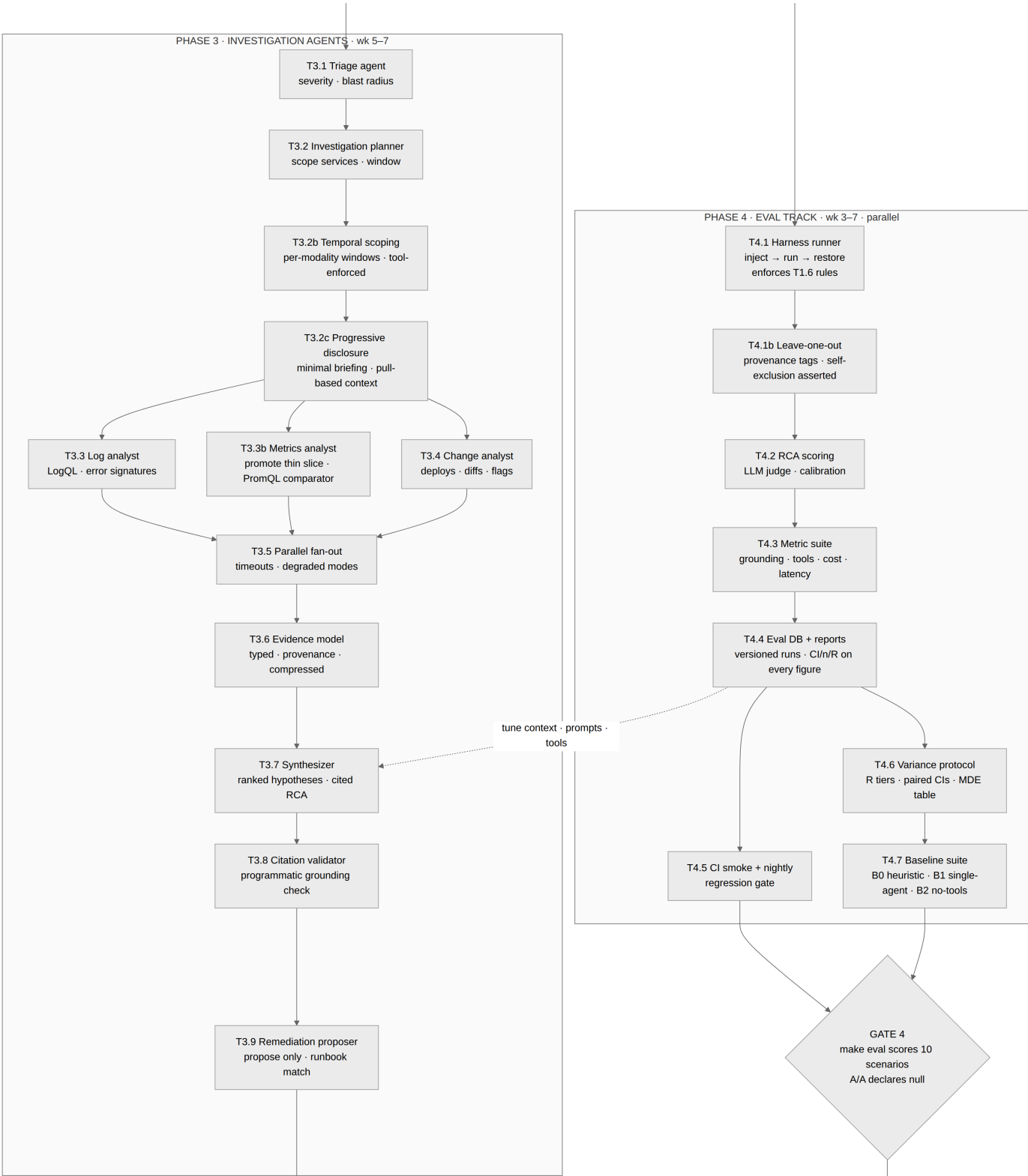
Weeks assume roughly 15–20 focused hours/week. Full-time, halve everything. The MVP milestone (end of P5) is when you start applying — P6/P7 continue while applications are out.

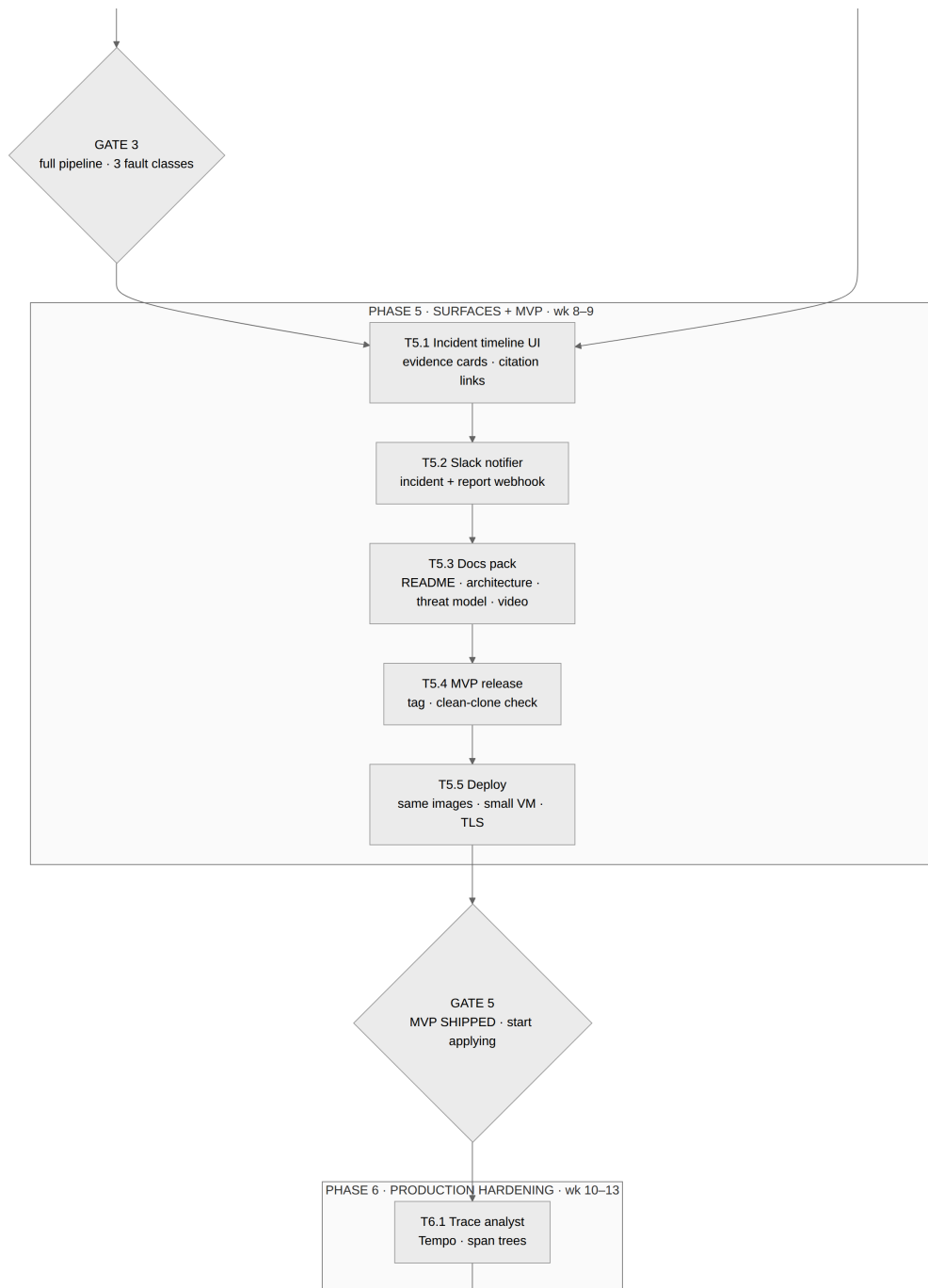
§2 Master flowchart

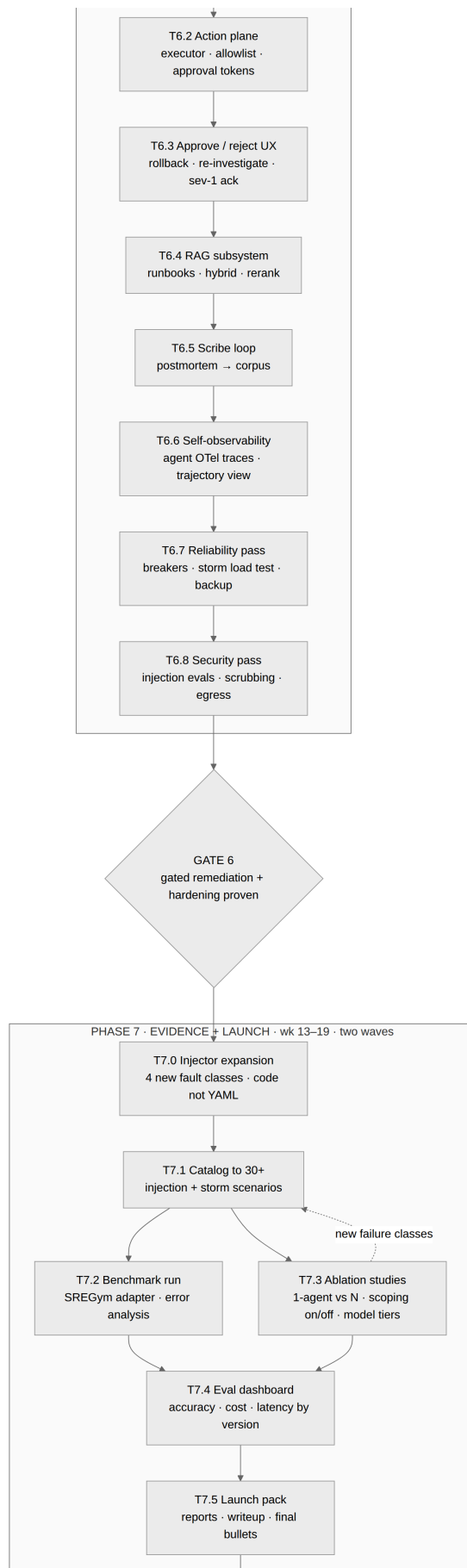
Every task in the project, in dependency order. Solid arrows are "must finish before"; dotted arrows are feedback loops — the mechanism that makes this an eval-driven build rather than a linear one. Diamonds are gates. Task IDs (T0.1, T3.7...) match the catalog tables in §3–§10.

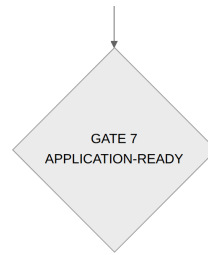












Two structural facts worth noticing: the eval track (P4) branches off as soon as the database exists (T2.3) and feeds back into the synthesizer (T3.7) — that dotted edge is the "evaluation → improvement → regression" loop the JD asks for, drawn as an arrow. And Gate 5 is the moment applications go out: P6 and P7 deepen the story while recruiters are already reading it.

§3 Phase 0 — Foundations WK 1

One week of discipline that pays for itself every week after. Nothing here is glamorous; all of it is what separates "production-oriented" from "script that grew."

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T0.1	Repo scaffold	Set up the monorepo: package layout for platform, injector, eval harness; dependency management; formatter, linter, strict type checking; pre-commit hooks.	Reviewers judge engineering maturity from the first minute of repo reading. Strict typing from day one is also what makes agent I/O contracts enforceable later.	Single repo, clear top-level module boundaries mirroring the architecture; type-checking configured strict from the start, not retrofitted.	Repo skeleton with green lint/typecheck; CONTRIBUTING notes
T0.2	CI pipeline	A pipeline that runs lint → typecheck → unit tests → container build on every push.	Every later phase adds a stage to this spine (integration tests, eval smoke). If CI arrives late, it arrives never.	Hosted CI with caching for fast runs; a deliberately trivial passing test proves the walking skeleton.	Green pipeline badge; build artifacts published per commit
T0.3	Compose base	Container orchestration files with profiles: platform-only, full-world, eval.	"Runs from clean clone with one command" is a gate condition for every phase — it must exist before there's anything to run.	Profile-based compose setup; documented make targets as the only entry points anyone ever types.	One-command bring-up for each profile
T0.4	ADR log	Start the architecture-decision-record habit: one short dated note per non-obvious choice.	The interview currency of this project is <i>defended decisions</i> — Redis Streams vs. Kafka, pgvector vs. dedicated DB, no agent framework. ADRs are where those defenses accumulate.	Numbered markdown records with context, decision, and consequences; first three ADRs written this week.	ARCHITECTURE.md + ADR directory with first entries

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T0.5	Benchmark spike	Clone the target benchmark harness (SREGym / ITBench-class), read its agent interface, and produce a written integration contract plus a go / adapt / no-go verdict, recorded as an ADR.	T7.2 is the highest-uncertainty node in the DAG and the one the most downstream claims depend on — the v2 identity, the second resume bullet — yet it had zero upstream validation and a plausible collision with the no-Kubernetes rule (these harnesses often ship their own cluster-based orchestration). It has no dependencies of its own, so nothing justified its risk sitting at the far end of the graph. Discovering incompatibility at week 13 invalidates a bullet already in circulation since Gate 5.	Answer four questions on paper: what does the harness hand an agent (cluster credentials? telemetry endpoints? its own tool API?); what does it expect back; what runtime does it require; can an external agent plug in at all? The contract feeds T2.6's design — environment handles injected, tool endpoints from config. If the verdict requires Kubernetes: an ADR carve-out (kind/k3d for the harness only, never for Faultline). If no-go: T7.2 is descope <i>d now</i> and the v2 bullets rewritten before they're ever used.	Integration contract + go/adapt/no-go ADR

GATE 0

CI is green on the walking skeleton; a clean clone brings up an (empty) platform with one command.

§4 Phase 1 — The World WK 1–2

Build the environment Faultline will investigate — a real polyglot microservices system with real telemetry, and a way to break it on command. Deliberately zero AI in this phase: if the world isn't real, nothing downstream is honest.

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T1.1	Deploy OTel demo	Stand up the OpenTelemetry Demo ("Astronomy Shop"): ~10 polyglot microservices plus its built-in load generator.	An incident platform without incidents is a mock. The demo gives realistic services, pre-built instrumentation, and continuous traffic for free — effort goes into the platform, not into inventing an app.	Pin a specific demo version; wire it into the compose profiles; verify steady-state traffic and baseline dashboards.	Running 10-service environment under load
T1.2	Telemetry backends	Route all signals to your own stack: metrics to Prometheus, logs to Loki, dashboards in Grafana. (Tempo/traces deferred to P6.)	Vendor-neutral positioning starts here: Faultline investigates the open Grafana stack, the segment Datadog structurally can't serve. These backends are also the agents' future tool endpoints.	Configure the OTel collector pipelines to the three backends; build one "shop health" overview dashboard used in every demo.	Prometheus + Loki + Grafana wired; health dashboard

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T1.3	Alert rules	Author alerting: error-rate, latency SLO burn, and saturation rules per service, routed through Alertmanager.	Alerts are Faultline's input signal. Writing them yourself teaches the exact semantics (for/labels/grouping) the triage agent must later reason about.	Small set of high-signal rules (not one per metric); Alertmanager grouping and a webhook receiver placeholder for P2.	Alert rules firing correctly on induced failure
T1.4	Chaos injector CLI	A command-line tool that injects and cleanly reverts faults across four classes: bad deploy, dependency latency, resource exhaustion, bad config.	This is the ground-truth machine — the single artifact that makes real evaluation possible, and the thing almost no other candidate will have.	Each fault is a named, parameterized, reversible operation against the demo environment; injection and restore are idempotent and logged.	Injector CLI with 4 fault classes, inject/restore verified
T1.5	Scenario catalog	A structured (YAML) catalog format: fault definition, true root cause, expected evidence, expected remediation class — and the first 10 labeled scenarios.	The catalog is simultaneously the eval dataset, the regression suite, and the demo script library. Its schema decides what "correct" means for the whole project.	Schema designed with the eval harness in mind (machine-checkable expectations); scenarios authored under T1.6's protocol — split assigned before any rehearsal happens — spread across the 4 fault classes; each scenario rehearsed end-to-end manually, its artifacts landing in split-labeled directories.	Scenario schema + 10 ground-truth labeled scenarios
T1.6	Holdout protocol	Declare the development / holdout split at scenario-creation time (~70/30, balanced across fault classes), with contamination rules: rehearsal artifacts of holdout scenarios are quarantined from the start, and no corpus, prompt, or context tuning may ever touch a holdout scenario. Headline accuracy is reported holdout-only.	The split must exist <i>before the first rehearsal artifact is produced</i> — T1.5's rehearsals feed T2.4b's past-incident store, so declaring the split any later (the original plan had it in Phase 4, then briefly <i>after</i> T1.5's rehearsals) lets holdout material leak into retrieval before the rule exists — which is why the flow now draws T1.6 before scenario authoring, not after it. Without this, headline accuracy is train-set accuracy and won't survive a competent interviewer.	Split recorded as a field in each scenario's YAML at authoring; rehearsal artifacts land in split-labeled directories so quarantine is mechanical; the eval harness (T4.1) later enforces split quarantine by checksum, and T4.1b enforces the second axis — run-time self-exclusion; the ADR records the two-axis contamination model (split quarantine + self-exclusion), which task enforces which, and the headline policy — full-set numbers with labeled split and explicit n until the catalog reaches 30+, holdout-only headline thereafter (a 3-scenario holdout is too small to headline).	Dev/holdout assignment in the catalog + quarantined artifacts + contamination ADR

GATE 1

Run one injector command → the right alert fires → the failure is visible on the Grafana dashboard. All deterministic, all yours, no AI anywhere yet.

The plumbing that turns an alert into a managed, persistent, observable investigation — plus a first thin slice through the whole system so integration risk dies early.

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T2.1	Ingest API	An async webhook endpoint receiving Alertmanager payloads; fingerprints each alert and dedupes repeats into a single incident.	Alert storms are the first real-world condition to survive: 200 firing alerts must become a handful of incidents, not 200 investigations.	Fingerprint from alert identity labels; uniqueness enforced at the database level so duplicate creation is structurally impossible; the FastAPI app's generated OpenAPI schema is the platform's public REST contract from day one.	Ingest endpoint with dedupe + documented OpenAPI surface
T2.2	Event bus	Decouple ingestion from investigation with Redis Streams: consumer groups, acknowledgments, pending-claim on worker death — plus the global investigation concurrency cap : at most N investigations run at once, overflow queues by severity.	Investigations are slow and expensive; webhooks must return instantly, and work must survive worker crashes. The cap is the spec's alert-storm cost defense: 200 alerts must degrade to queueing, never to 200 parallel LLM bills. (Streams over Kafka — an ADR, and a prepared interview answer.)	Consumer-group semantics with idempotent handlers keyed on incident + step; heartbeat and reclaim for crashed workers; the cap enforced at the orchestrator boundary, overflow drained severity-first from the stream — T6.7's storm load test exercises exactly this path.	Durable event flow + global concurrency cap with severity-ordered overflow; kill-a-worker test passes
T2.3	State machine + DB	Postgres schema for incidents, agent runs, evidence, and audit entries; an explicit eleven-state incident machine — detected → triaged → investigating → hypothesis → awaiting-approval → remediating → resolved, plus duplicate-merged (alert-storm and two-workers rows), budget-exhausted (runaway-cost row, re-enterable when the cap is raised), rejected (wrong-root-cause row, exits to targeted re-investigation, reason required), and failed (terminal, e.g. persistent citation-validation failure) — with migrations. Acceptance: every row of the spec's failure-scenario table names a reachable state.	All agent state lives in the database, not in process memory — that's what makes investigations resumable, replayable, and auditable, and it's the difference between an agent system and an agent script.	Transitions validated in one place; every write idempotent; property-based tests on legal and illegal transitions across all eleven states; integration tests running real Postgres and Redis via testcontainers; migrations from day one; raw evidence payloads and rendered reports archived to S3-compatible object storage so citations stay re-verifiable forever.	Schema + migrations + tested state machine + report/evidence archive

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T2.4	Service catalog	A git-versioned catalog of the demo's services: owners, tiers, SLOs, runbook links, declared dependencies — exposed as a dependency-graph API.	This is the context layer's foundation. Topology scoping (alert on checkout → investigate checkout ± 1 hop) filters irrelevant services <i>structurally</i> , before any model sees anything.	Catalog as YAML in the repo; graph traversal API with "blast radius of service X" as the core query.	Catalog + dependency graph with scoping API
T2.4b	Knowledge stubs	Seed the minimal knowledge stores Phase 3 consumes: ~15 runbooks with basic hybrid (dense + keyword) search, a past-incident store initially filled from scenario rehearsals, and the allowlist action catalog as a read-only versioned document.	The planner (T3.2) needs "similar past incidents" and the proposer (T3.9) needs runbooks and an allowlist — none of which would otherwise exist before Phase 6. Deliberately minimal: T6.4 replaces the basic search with the full retrieval pipeline; T6.2 makes the allowlist executable.	Runbooks authored alongside T1.5's scenarios so corpus and catalog grow together — but seeded only from the T1.6 development split ; holdout scenarios and their rehearsal artifacts never enter these stores, and every seeded record is stamped with its provenance (<code>origin: authored scenario:<id></code>) so T4.1b's exclusion filter has something to filter on. No reranking, no ingestion pipeline yet — just enough retrieval to build honest agents against.	Seeded runbook corpus + past-incident store + read-only allowlist
T2.5	LLM gateway	One choke point for all model calls: provider routing, retries with backoff, timeouts, per-incident token/dollar budgets, cost metering.	Model outages and rate limits are the most common runtime failure; budgets are the difference between a \$0.60 investigation and a \$40 loop. One seam also enables P7's model-tier ablations.	Thin gateway layer (own adapter or a lightweight router) with structured logging of every call: model, tokens, latency, cost, incident. The seam is proven provider-agnostic against an OpenAI-compatible endpoint — the self-hosted vLLM lane from the positioning, recorded as an ADR.	Gateway with fallback, budget enforcement + verified self-hosted seam
T2.6	Agent runtime	The in-house agent loop (~few hundred lines): typed tool contracts, schema-validated structured outputs with bounded re-ask, per-agent token and tool-call budgets, full trajectory persistence — and the injection defense <i>built in</i> : every tool result is wrapped as delimited, typed, trust-labeled data (telemetry is untrusted input), and privileged decisions (state transitions, action proposals) are validated outside the model.	Deliberately no heavyweight agent framework — the runtime is exactly the skill this JD pays for, and owning it makes every later behavior (budgets, degraded modes, replay) tractable. Another flagship ADR.	Tools declared as typed interfaces, each holding its own scoped read-only credential — no shared god-token on the read plane; every agent step writes its inputs, outputs, tokens, and cost to the runs table as it happens; the runtime ships a record/replay harness so agents are unit-tested against recorded LLM fixtures.	Reusable runtime with trust-labeled tool I/O, per-tool credentials, fixture harness + benchmark-compatible entry point per T0.5's contract (environment handles injected, tool endpoints from config — nearly free now, expensive to retrofit at week 13)

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T2.7	Thin slice	One agent (metrics analyst) wired end-to-end: alert in → PromQL tool calls → typed findings persisted → rendered in a minimal report.	Integration risk dies in week 4, not week 8. The thin slice proves ingest, bus, state machine, gateway, and runtime cooperate before seven more agents pile on.	Deliberately narrow: one agent, one tool, one fault class, ugly-but-real output.	First end-to-end investigation of a real injected fault

GATE 2

Inject a fault → alert lands → an agent investigates → a persisted, rendered finding exists in the database and on screen. The eval track (§7) is already running in parallel — it branched at T2.3, mid-phase — so from here every agent decision is made against eval numbers. And the machine must express its own failure table: every failure-scenario row names a reachable state, property-tested across all eleven — a state machine validated against a failure table it cannot express is a test suite validating the wrong artifact.

§6 Phase 3 — Investigation agents WK 5–7

The multi-agent heart: parallel specialists gather typed evidence, a synthesizer builds cited hypotheses, and a validator makes hallucination a hard failure. Every design choice here gets made against eval numbers from the parallel track.

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T3.1	Triage agent	Classify each incoming incident: severity, affected service, blast radius, duplicate-of, suspected fault category — as a validated structured output driving the state machine.	Cheap and fast, it runs on everything, gating whether an expensive investigation launches at all. It's also the natural home for the cheap-model routing tier.	Small model, strict output schema, service-catalog, dependency-graph, and alert-history lookups as its only tools (blast radius needs the graph) — T2.1's fingerprint dedupe owns exact duplicates; triage's duplicate-of catches cross-fingerprint correlation, which is why it needs the history; wrong-but-cheap beats slow-but-perfect here, measured by eval.	Triage decisions persisted; noise gated before fan-out

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T3.2	Investigation planner	Turn a triaged incident into a plan: which specialists, over which topology-scoped services, in what time window, with what initial hypotheses.	Scoping decisions must happen once, centrally — otherwise every specialist re-derives them and the fan-out loses its point.	Consumes the dependency graph and top similar past incidents (from the T2.4b seeded store); emits a typed plan the orchestrator executes mechanically.	Plan objects; scoped fan-out replaces investigate-everything
T3.2b	Temporal scoping	Make time a first-class context filter: every tool derives its default window from alert onset (onset – 30 min → now), the change analyst alone widens its lookback (onset – 24 h, because causes precede symptoms), and the planner may widen a window per hypothesis — all enforced at the tool layer, never left to agent discretion.	One of the spec's four named context-engineering mechanisms — and unbounded time ranges are how log agents drown. A mechanism that exists only as a clause inside another task is one the repo can't demonstrate and no number can defend; given context engineering is the JD's first bullet, it needs an owner.	Window policy lives in the tool contracts; every query logs its window; a request for an unbounded range is rejected with a narrowing hint the agent must act on.	Tool-enforced window policy + per-query window logging
T3.2c	Progressive disclosure	Agents start from a minimal briefing — the alert, scoped topology, top-3 similar past incidents — and <i>pull</i> further context via tools on demand, rather than receiving everything push-style; briefing size is token-budgeted per role and measured.	The fourth named mechanism, and the one the JD's "right context at the right time" phrase literally describes. The difference between context engineering and prompt stuffing is that context arrives on demand — this task is where that claim becomes code.	A briefing assembler builds the per-role pack under budget; everything else is tool-reachable; briefing size and pull-rate are logged per run so T7.3's ablation has data to compare against.	Budgeted briefing assembler + pull-rate metrics
T3.3	Log analyst	Specialist over Loki: query scoped services/windows, extract error signatures, first-occurrence times, correlated bursts.	Logs are the highest-signal, highest-volume modality — the one that most needs its own context budget and result-shaping to avoid drowning.	Query tool validates LogQL and caps result sizes; agent returns compressed typed evidence, never raw log dumps.	Log evidence with signatures + sample lines + provenance
T3.3b	Metrics analyst	Promote T2.7's thin-slice prototype into the full specialist the spec's agent table describes: baseline range-query comparison (incident window vs. normal), saturation / error-rate / latency anomaly extraction, change-point timestamps.	The thin slice proved plumbing, not analysis. Resource-exhaustion faults — including the scripted recruiter-demo scenario — are fundamentally metrics stories; without this promotion, Gate 3's "3 of 4 fault classes" would quietly become "the three that aren't saturation."	The PromQL tool gets the same parse-validate-and-cap treatment as LogQL (query-language sandboxing parity); baseline-comparison queries templated; standard typed evidence out.	Production metrics specialist; demo fault class fully covered
T3.4	Change analyst	Specialist over change sources: recent deploys, config changes, feature flags, git diffs near the incident window, ranked by suspicion.	Most real incidents are change-induced; "what changed?" is the single highest-yield investigative question, so it gets a dedicated agent with a wider lookback window.	Deploy-history and repo-compare tools; time-proximity plus blast-radius ranking of candidate changes.	Ranked suspicious-change evidence per incident

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T3.5	Parallel fan-out	Orchestrate specialists concurrently with per-agent timeouts and budgets; a failed specialist degrades the investigation rather than aborting it.	The fan-out is both the latency win and the justification for multi-agent design; graceful degradation is what makes it production-shaped.	Async concurrency in the orchestrator; "modality unavailable" recorded as typed evidence the synthesizer must acknowledge.	Concurrent investigations; kill-one-specialist test passes
T3.6	Evidence model	The typed Evidence object: claim, source query, time range, raw-result hash, sample payload — the only currency agents may exchange.	Evidence-as-compression is the project's core context-engineering move: the synthesizer sees a curated evidence board, not transcripts. It's also what makes citations checkable.	Strict schemas per modality; provenance mandatory; every Evidence object carries the trust label its source tool attached in the runtime; raw results archived so every citation can be re-verified later.	Evidence store with full provenance chain
T3.7	Synthesizer	Consume all evidence; produce ranked causal hypotheses and the RCA report, every claim carrying evidence IDs; optionally dispatch one targeted follow-up round.	Synthesis genuinely needs all evidence — the one legitimate barrier in the pipeline — and it's where the strongest model earns its cost.	Frontier model, structured report schema, explicit alternative hypotheses with confidence; follow-up round strictly bounded.	Cited, ranked RCA reports
T3.8	Citation validator	A deterministic checker: every cited evidence ID must exist, and quoted content must match the stored raw result; violations block the report.	This converts "how do you handle hallucination?" from a hand-wave into a mechanism: a fabricated log line is a hard failure, counted and charted.	Pure non-LLM validation at the publication boundary; failures feed back for one regeneration, then page a human.	Grounding enforcement gate + violation metrics
T3.9	Remediation proposer	Map the accepted hypothesis to runbook-derived remediation options with risk notes — strictly propose-only in this phase.	Keeps the propose/execute boundary architectural from the start, so P6's action plane slots into an existing seam instead of retrofitting safety.	Proposals reference the T2.4b allowlist catalog and seeded runbooks (read-only until T6.2 makes actions executable); risk and blast-radius notes mandatory.	Remediation proposals attached to RCA reports

GATE 3

The full pipeline — triage, plan, parallel specialists, synthesis, validated citations, proposal — completes successfully on at least 3 of the 4 fault classes.

§7 Phase 4 — Evaluation harness

WK 3–7 · PARALLEL

The parallel track that makes everything else honest. It starts the moment the database exists (week 3) and grades every agent decision made in Phase 3 — this ordering is the plan's most important rule.

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T4.1	Harness runner	An automated loop per scenario: inject the fault → wait for the alert → run Faultline end-to-end → restore the environment → collect the artifacts.	Manual testing of a probabilistic system is theater. The runner turns "does it work?" into a repeatable experiment over the ground-truth catalog.	Drives the injector and platform through their public interfaces only; hermetic per-scenario cleanup verified between runs; enforces T1.6's contamination rules mechanically — refuses to score a holdout scenario whose artifacts appear (by checksum) in any corpus.	One-command scenario runner producing raw run artifacts
T4.1b	Leave-one-out enforcement	The contamination model's second axis: provenance tagging on every knowledge artifact (<code>origin: authored scenario:<id></code>) plus a run-time exclusion filter — a scenario's own artifacts are unreachable from retrieval while that scenario is being scored — which the harness sets and then <i>asserts fired</i> .	Without it, dev-set accuracy measures retrieval of the answer key: when scoring scenario S, the nearest neighbor in the past-incident store is the rehearsal of S, containing its true root cause in the label author's words. Split quarantine (T1.6) doesn't catch this — it's within-split self-reference, and it's the kind of finding an interviewer makes in thirty seconds by reading the seeding code.	Hand-authored runbooks stay <code>authored</code> and are never excluded — they're legitimate institutional knowledge. Retrieval tools accept <code>exclude_origin</code> ; T4.1's runner sets it to the scenario under test on every scored run; the count of filtered artifacts is logged per run, and a scored run where the filter did not fire is marked invalid , not merely annotated — silent non-enforcement is how this defect returns.	Provenance field across all stores + enforced exclusion with per-run filter counts + invalid-run detection
T4.2	RCA scoring	Score each run against the scenario's labels: root-cause top-1 and top-3 accuracy (LLM judge, semantic equivalence), time-to-first-correct-hypothesis, and remediation-class correctness against T1.5's labeled remediation field.	Root-cause correctness is the headline metric, and judge calibration (agreement rate with your spot-audits) is what makes it credible in an interview.	Judge with a strict rubric and structured verdicts; ~30 manually graded runs establish the agreement baseline before trusting it.	Accuracy scores + documented judge-calibration audit
T4.3	Metric suite	The non-headline metrics: citation validity (programmatic), tool-call validity rate, trajectory efficiency (steps, redundant calls), per-agent context-budget-overflow rate, latency, and dollar cost per investigation.	These are the metrics that expose <i>why</i> accuracy moves — and they're mostly deterministic, so they anchor the eval against judge noise.	Computed from persisted trajectories and gateway logs; no new instrumentation needed because P2 recorded everything.	Full metric panel computed per run

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T4.4	Eval DB + reports	Persist every eval run with its config fingerprint (prompt versions, model, context settings); generate a comparative report between any two versions.	This is the hill-climbing ledger: "evidence-compression refactor took top-1 from 61% to 79%" only exists if versions are tracked. It also becomes P7's ablation engine for free.	Eval schema alongside the platform DB; config captured as a hash of all behavior-relevant settings — including repeat count R, judge version, and seed policy (T4.6); the report generator emits mean, 95% CI, n, and R on every figure it prints, and a figure without all four is a bug in the generator; reports break out dev vs. holdout — headline numbers follow T1.6's policy: full-set with labeled split and explicit n until the catalog reaches 30+, holdout-only thereafter.	Versioned eval history + comparison reports
T4.5	CI smoke + nightly	Wire evals into CI: a small smoke subset on every change touching prompts/context/models; the full catalog nightly with trend tracking.	This is the "regression testing loop" of the JD, literally: a prompt edit that tanks accuracy fails the build instead of shipping silently.	Smoke suite chosen for coverage-per-minute; nightly results appended to the eval DB; regression thresholds fail loudly; smoke results are labeled <i>non-citable</i> in the CI output itself (R=1 is change detection, never a finding), so a smoke number can't be screenshotted into a README six weeks later.	Eval-gated CI + nightly trend line
T4.6	Run-variance protocol	Repeat counts, paired comparison, and confidence intervals as first-class properties of every eval run and every comparison report — plus a pre-computed minimum-detectable-effect table checked into the repo before the first ablation runs.	The plan reported point estimates from single runs of a stochastic system, then built resume bullets on the deltas. At n=30 with one run per config, sampling noise is roughly $\pm 9pp$ — wide enough to swallow most of the ablation series. "How many runs, and what's your variance?" is the first question a competent reviewer asks of an ablation table; a number without an interval is a vibe with extra steps.	Repeat count R joins T4.4's config fingerprint. Three tiers: CI smoke R=1 (change detection only, never citable), scored comparisons R=5, nightly R=1 with weekly R=3 consolidation. Comparisons are <i>paired</i> — same scenarios, same seeds where seedable, bootstrap over per-scenario deltas — worth roughly $2\times$ effective sample size for free. MDE table: 10 scenarios $\approx 20pp$ (directional only, never publish a number); $30 \approx 10pp$; 30 paired at R=5 $\approx 6-7pp$ (most of T7.3 becomes citable). A delta below its MDE is reported as "no measurable effect at this catalog size" — stronger interview material than a fabricated 3-point win. Cost flagged, not hidden: R=5 is a $5\times$ spend multiplier, confined by the tiering to comparisons that get published.	Variance protocol in the eval schema + CI-bearing reports + MDE table in evals/

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T4.7	Baseline suite	Three permanent baselines scored by the same judge, at the same R, on the same catalog, appearing in every headline table: B0 — a no-LLM heuristic (alert-label attribution + most-recent deploy in window + largest error-rate delta → top-1); B1 — single agent with all tools, promoted from a one-off ablation to a permanent baseline; B2 — frontier model with alert text and service catalog but no tool access, isolating how much accuracy comes from the model's prior rather than the investigation.	Absolute accuracy is uninterpretable — 84% top-1 is only a result relative to something, and if a fifty-line heuristic reaches 70%, the multi-agent claim is much weaker. You need to be the one who discovers that. Baselines pre-empt the sharpest available attack ("is any of this doing anything?") by answering it in your own README.	All three run under the standard harness as ordinary configs in the eval DB. The MTTR claim gets its missing left-hand side the honest way: self-timed manual RCA on five dev scenarios, reported as <i>n=5, self-timed, indicative</i> — an unsourced number next to a rigorously sourced one damages the rigorous one.	B0/B1/B2 configs in the eval DB + baseline columns mandatory in every headline table + measured manual-RCA reference

GATE 4

make eval injects, runs, scores, and reports all 10 scenarios unattended; CI blocks a deliberately-broken prompt change; the latency/cost thresholds hold — dev-set median time-to-report ≤ 3 minutes, cost ≤ \$2 per incident; and the **A/A check passes**: the harness run twice under an identical config declares no significant difference. A harness that invents a delta between a config and itself will invent every delta it ever reports — this is the cheapest possible proof the instrument is calibrated. The MTTR claim as a falsifiable number, not a demo anecdote.

Deliberately cut, not forgotten: the spec's online-evaluation block — per-incident thumbs, sampled judge scoring of live reports, canary drift alarms — is deferred beyond this plan, because online signals only mean something under sustained real usage a portfolio system doesn't have. The seams exist (every report, approval, and rejection is already persisted), and an ADR records the cut so it reads as a scoping decision, not an omission.

§8 Phase 5 — Surfaces & MVP ship WK 8–9

Make the system visible, documented, and demoable — then tag it and start applying. The MVP is a complete story: world, investigation, grounding, evaluation.

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T5.1	Incident timeline UI	A focused web view per incident: live-updating timeline, evidence cards per specialist, the ranked RCA report, and citations that deep-link into Grafana queries.	The clickable citation is the demo's most convincing moment — a claim that lands you in the actual data. One great screen beats five mediocre ones.	Minimal single-page frontend over the platform API; live updates via polling or server-sent events; no design system, just clean hierarchy.	Incident view with evidence cards + citation deep-links
T5.2	Slack notifier	Webhook notifications on incident open and report ready, with links into the UI.	Signals "this lives where on-call actually lives" at near-zero cost; full Slack-app interactivity is explicitly out of scope.	Plain incoming-webhook messages; formatting kept minimal.	Incident lifecycle notifications
T5.3	Docs pack	README with the demo video and eval table up top; ARCHITECTURE.md with the system diagram and ADR index; a threat-model document; a 4-minute recorded demo; and the MVP-cut resume bullets — claims scoped to what exists at Gate 5 (logs, metrics, and deploy history; remediation as proposals with risk notes) — used in applications until T7.5 swaps in the v2 set.	Reviewers read documents before code. Most candidates have none of these four; together they're what makes "production-oriented" credible unopened.	Written like internal engineering docs, not marketing; the video records the MVP cut of the spec's demo script — inject, live investigation, cited report, remediation as proposal, 10-scenario eval table. (The full cut — approved rollback, Tempo encore, 30-scenario dashboard — is re-recorded in T7.5, once its beats exist.) The README's eval table shows all 10 scenarios with dev/holdout labels and n stated explicitly, per T1.6's headline policy — the holdout-only headline switches on when T7.1 brings the holdout to ~9–10 scenarios.	README · ARCHITECTURE · THREAT-MODEL · demo video · MVP-cut bullets
T5.4	MVP release	Tag v0.1; verify the whole story — world up, fault injected, investigation, eval run — from a clean clone on a fresh machine.	"Works on a stranger's machine" is the only definition of shipped that matters for an open project.	Clean-clone rehearsal following only the README; fix every papercut it reveals.	Tagged, reproducible MVP
T5.5	Deploy	Deploy the MVP: the same images CI builds, running continuously on a single small VM — demo environment plus platform, TLS in front, basic auth on the UI.	"Deployable" is only a claim until something is deployed. A live instance gives applications a URL to point at, exercises the local → CI → deployed path end-to-end, and makes the deferred online-eval seams real infrastructure rather than hypothetical ones.	Compose profiles on the VM, images pulled from the registry CI publishes to; secrets via env; an uptime check; a documented deploy-and-rollback procedure rehearsed once. Size honestly: the full stack (demo world + telemetry + platform) wants ~4 vCPU / 16 GB — the OTel demo alone needs ~6 GB — so either budget for that or run the cheaper option, platform plus a trimmed demo profile (the OTel demo's compose flags support service subsets); the choice and its monthly cost go in the deploy doc.	Live instance at a stable URL + documented deploy procedure

GATE 5 • MVP SHIPPED

The full demo runs from a clean clone, and a deployed instance is live at a stable URL. **Applications start going out now** — Phases 6–7 deepen the story while it's already working for you.

§9 Phase 6 — Production hardening WK 10–13

The MVP investigates; this phase makes it *act* — safely — and makes the platform itself observable, reliable, and security-tested.

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T6.1	Trace analyst	Add Tempo to the telemetry stack and a fourth specialist that finds exemplar slow/failed traces and identifies the degrading hop in the request path.	Traces answer the question logs and metrics can't: <i>where in the path</i> . Completes the four-modality investigation story.	Trace-search and span-tree summarizer tools; evidence model already accommodates the new modality.	Trace evidence in investigations; eval accuracy delta measured
T6.2	Action plane	A separate executor service holding the only write credentials: allowlisted, parameter-validated actions (rollback, restart, flag toggle, scale) executed only with a valid single-use approval token; append-only audit; kill switch.	The flagship safety mechanism: even a fully prompt-injected investigation agent cannot execute a write, because the tokens it holds cannot. Structural security, not prompt-level.	Process separation with distinct credentials; action-bound tokens minted at approval time; blast-radius check re-validates the target against incident scope; every executed action records its inverse where one exists, enabling one-click revert.	Executor service + audit log + kill switch
T6.3	Approve / reject UX	The human loop, both directions: approve a specific proposed action from the UI or Slack link and watch execution and recovery on the timeline — or reject a hypothesis or report, which triggers a targeted re-investigation and auto-drafts the miss as a dev-set eval scenario. Severity-1 reports require human acknowledgment before broad distribution.	Human-in-the-loop is only real if disagreement is a first-class path; the failure table's "wrong root cause confidently reported" row recovers through rejection, and rejection is the retained half of the spec's deferred online-eval loop.	Approval mints the token; rejection captures a required reason that seeds both the re-investigation brief and the drafted scenario; execution status and post-action metric snapshots land back on the incident timeline.	Approved remediation with visible recovery + rejection → re-investigation loop + sev-1 ack gate

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T6.4	RAG subsystem	Ingest runbooks, postmortems, and service docs (git-synced): heading-aware chunking with document-level summaries, metadata, hybrid dense+sparse retrieval, cross-encoder rerank, cited excerpts in agent context — growing the corpus from T2.4b's 15 seeds to ≥50 documents — the stated size the retrieval gates are measured against.	Retrieval where retrieval belongs: institutional knowledge, not live telemetry. Gives the synthesizer and proposer memory of "what worked last time."	pgvector + full-text fusion; embeddings behind a swappable interface; ranking is recency-aware and deprecated docs are excluded by metadata filter at ingest; the metadata filter chain carries origin alongside service and recency so T4.1b's self-exclusion applies here too; retrieval golden set (~50 query → doc pairs) with recall@5 and MRR gated in CI.	Retrieval pipeline over a ≥50-doc corpus + recall@5 / MRR gates
T6.5	Scribe loop	A post-resolution agent drafting the postmortem from the incident record; accepted postmortems join the retrieval corpus.	Closes the knowledge loop — and enables the flagship "does the system learn?" eval: measured as cross-scenario transfer within a fault class — hold scenario S, seed the corpus with postmortems from <i>other</i> scenarios in S's class, compare against a corpus without them. Same-scenario "N+1 beats N" was memorization by construction (the corpus held a postmortem written from the earlier run of the same scenario) and is unfalsifiable under T4.1b's self-exclusion anyway; a positive transfer delta means the system genuinely generalizes from prior incidents — the thing worth claiming, now the thing that can be defended.	Draft-for-human-edit, never auto-published; corpus growth tracked in the retrieval evals; postmortems generated from eval runs are quarantined under T1.6's contamination rules, and the transfer measurement uses dev-set fault classes only, with the with/without-corpus comparison run at T4.6's scored-comparison tier, so holdout scores stay clean.	Postmortem generation + learning-effect measurement
T6.6	Self-observability	Instrument Faultline itself with OpenTelemetry: a span per agent step, tool call, and model call, exported to the same Tempo/Grafana as the demo app; structured JSON logs; Prometheus metrics for queue depth, investigation latency, and eval scores; a trajectory view in the UI.	The agent's own trace beside the outage it's investigating is the single most memorable engineering demo beat — and it's real agent observability, a JD line item.	Span attributes carry incident, agent, model, tokens, cost; one Grafana dashboard for platform health and spend — queue depth is the metric that makes T2.2's concurrency cap observable, which is why it's named rather than left to "misc metrics."	Agent traces in Grafana + platform metrics & JSON logs + trajectory UI
T6.7	Reliability pass	Circuit breakers on providers and tools, alert-storm load test at 200+ alerts, worker-crash chaos test, DB backup/restore drill, timeout audit.	The failure-scenario table in the spec becomes tested behavior instead of claimed behavior — each row gets a test or a drill.	Storm replay harness; induced worker kills mid-investigation; documented restore procedure actually executed once.	Passing storm/crash tests + restore runbook

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T6.8	Security pass	Prompt-injection scenarios (malicious log lines, poisoned commit messages) added to the catalog; secret scrubbing before model calls; egress restriction from the agent runtime; secrets management tidy-up; and re-hardening of the deployed instance's public surface now that the action plane exists — approval endpoints authenticated, the executor unreachable from the internet (private network only), the kill switch reachable to the operator, the exposed surface re-reviewed before Gate 6.	"Telemetry is untrusted input" is the project's distinctive security thesis — T2.6 <i>builds</i> the defense (trust-labeled wrapping, out-of-model validation); this task attacks it and proves it holds.	Injection scenarios scored in the standard eval loop; network policy limits runtime egress to gateway + declared tools.	Injection eval results + documented defenses

GATE 6

An approved remediation executes and recovers the system; injection and storm scenarios pass; the platform's own traces are on the dashboard; and the Gate 4 thresholds are **re-asserted with the full pipeline** — four specialists, retrieval, rerank, and self-instrumentation included — still holding median time-to-report ≤ 3 minutes and $\leq \$2$ per incident on the dev set. Latency budgets that are only checked once are checked never.

§10 Phase 7 — Evidence & launch WK 13–19 · TWO WAVES

The v2 positioning phase: convert a working system into public, measured evidence. This is the heaviest stretch in the plan — expanding the injector and rehearsing 20 new scenarios is comparable to all of Phase 1 — so it ships in **two waves**: wave one (wk 13–16) is injector expansion, catalog growth, benchmark run, and ablations; wave two (wk 16–19) is the dashboard, writeup, full demo cut, and launch. If time compresses, triage by artifact value, not by wave: the benchmark run (T7.2) and the writeup (T7.5) — the number, and the thing that carries the number to readers — outrank the ablation series and the dashboard. A launch with a benchmark report and no dashboard beats the reverse — that cut compresses this phase to ~3 weeks and the plan to its ~16-week floor; the full plan runs 19.

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T7.0	Injector expansion	Extend the chaos injector with the four new fault classes the expanded catalog needs — cert expiry, cache stampede, disk fill, N+1 regression — as real injector code with clean inject/restore, not YAML entries.	T7.1's "30+ scenarios across ~8 classes" doubles the injector's fault library; leaving that inside "catalog growth" silently doubles T7.1's scope, and the deliverables checklist already promises 8 classes. Making it a task makes the effort visible and schedulable.	Same reversibility and idempotence bar as T1.4; each new class rehearsed end-to-end before any scenario is authored against it.	Injector at 8 fault classes, inject/restore verified
T7.1	Catalog to 30+	Grow the scenario catalog to 30+ across ~8 fault classes, including the injection and storm cases from P6.	Statistical credibility: accuracy claims over 10 scenarios are an anecdote; over 30+, a measurement. Also raises the ceiling for ablation sensitivity.	New classes prioritized by realism (cert expiry, cache stampede, N+1 regression, disk fill...); each rehearsed and labeled like the first ten, and assigned to the dev or holdout split at creation per T1.6's protocol.	30+ scenario catalog, the project's headline dataset
T7.2	Benchmark run	Build a thin adapter presenting Faultline's agent to an external benchmark (SREGym / ITBench-class) and run it; publish a results report centered on per-scenario-class error analysis.	The heart of v2: numbers on a public yardstick where frontier baselines fail most incidents. A mid-tier score rigorously explained beats an unexplained high one.	Implements the integration contract T0.5 produced — a known interface, not a discovery exercise, dropping this task's risk from unbounded to ordinary engineering; adapter kept thin so the agent under test is the real one; report structured as scores → error taxonomy → architecture implications.	External benchmark report with error analysis
T7.3	Ablation studies	Controlled experiments on your own harness: single-agent-all-tools vs. parallel specialists; topology scoping on/off; evidence compression vs. raw context; model tier per role; the retrieval series — on/off, hybrid vs. dense-only, rerank on/off; temporal scoping on/off (unbounded windows); and progressive disclosure vs. push-everything briefings.	The study commercial vendors can't publish, and the direct proof of the JD's "hill-climbing methodology." Each ablation is nearly free because T4.4 versioned everything.	One config dimension varied at a time over the full catalog, at R=5 paired per T4.6, with run order randomized across configs to avoid confounding with provider-side drift; every report states its MDE <i>before</i> its result, and a below-MDE delta is reported as "no measurable effect at this catalog size"; dated experiment reports checked into the repo.	Ablation report series — the architecture-science story
T7.4	Eval dashboard	A small dashboard over the eval DB: accuracy, grounding, cost, and latency across prompt/model versions and scenario classes.	The recruiter demo's closing beat ("here's the measured hill-climb") and your own daily instrument for what to fix next.	Charts straight off the eval schema; version timeline annotated with what changed.	Versioned eval dashboard

ID	Task	What we're doing	Why we're doing it	How we'll do it	Deliverable
T7.5	Launch pack	Final README polish, the benchmark + ablation reports linked up top, the full demo cut re-recorded (approved rollback and recovery, Tempo encore, 30-scenario dashboard — replacing the MVP cut as the README's lead video), a technical writeup (blog-style) of the three most interesting findings, resume bullets updated with real numbers.	The project's reach now depends on writing: the writeup is what gets shared, and shared is how a portfolio project finds the people who hire.	Writeup leads with findings, not features; bullets follow the spec's v2 formulas with your measured values substituted in.	Public launch pack + final resume bullets

GATE 7 · APPLICATION-READY

Repo, video, benchmark report, ablation series, and dashboard together answer every question §5 of the spec says interviewers will ask — with numbers.

§11 Master deliverables checklist

Everything the project hands over, grouped by kind. When all boxes are checkable, you're done in the only sense that matters.

Running systems

- 10-service instrumented demo environment under continuous load
- Prometheus · Loki · Tempo · Grafana telemetry stack with alert rules
- Chaos injector CLI — reversible faults across 8 classes
- Faultline platform: ingest, event bus, eleven-state incident machine (property-tested against every failure-table row), agent runtime
- Nine agent roles — triage, planner, four parallel investigation specialists, synthesizer, remediation proposer, scribe — plus the deterministic citation validator and action executor (deliberately not agents)
- Action plane: allowlisted executor, approval tokens, audit, kill switch
- RAG subsystem over runbooks and postmortems, with Scribe loop
- Incident timeline UI with citation deep-links; Slack notifications
- One-command bring-up for every profile, from a clean clone
- Test infrastructure: record/replay LLM fixture harness + testcontainers integration suite
- Live deployed instance (same images as CI) at a stable URL

Evaluation assets

- 30+ scenario ground-truth catalog (the headline dataset)
- Automated harness: inject → run → score → restore, one command
- Two-axis contamination enforcement: split quarantine + per-run self-exclusion, filter counts logged, invalid-run detection
- Variance protocol: repeat tiers, paired comparisons, MDE table checked into evals/
- A/A calibration result (identical configs → declared null) at Gate 4
- Baseline suite B0 / B1 / B2 scored on every headline table + measured manual-RCA reference (n=5, self-timed)
- Benchmark integration contract + go/adapt/no-go ADR (produced in Phase 0)
- Cross-scenario transfer measurement for the learning claim (replaces same-scenario N+1)
- Judge-calibration audit with documented agreement rate
- Eval-gated CI + nightly full runs with trend history
- Retrieval golden set with recall@5 / MRR gates
- External benchmark report with per-class error analysis
- Ablation report series (agent count, topology + temporal scoping, progressive disclosure, compression, model tiers, retrieval series)
- Versioned eval dashboard: accuracy · grounding · cost · latency

Documents & media

- README with video and eval table first
- ARCHITECTURE.md + numbered ADRs (every defended decision)
- Threat-model document ("telemetry is untrusted input")
- Two recorded demo cuts: MVP cut at Gate 5, full cut at launch
- Technical writeup of the three most interesting findings
- Resume bullets: MVP-cut set at Gate 5, v2 set with measured numbers at launch

§12 Working rules

Six rules that keep four-plus months from drifting. **Eval before opinion:** from Gate 2 onward, no prompt, context, or architecture change lands without an eval comparison — the habit is the resume line. **The demo always works:** the scripted demo path is a protected invariant; any commit that breaks it gets fixed before anything else, because you never know when a recruiter clicks the repo. **Every decision gets an ADR:** if you had to think about it, write it down — interviews are won on these. **Gates are hard:** a phase isn't done because its code exists; it's done when its gate passes from a clean clone. **Scope is a feature:** the spec's do-not-build list (no Kubernetes for the platform, no Kafka, no multi-tenancy, no chat UI, no fine-tuning, no heavyweight agent framework) is part of the design — when tempted, add a line to the extensions section of the spec instead of code to the repo. **No number without an interval:** any figure that leaves the repo — README, writeup, resume bullet, interview answer — carries its n, its R, and its confidence interval, and sits

next to a baseline; a delta below its stated MDE is reported as no measurable effect. This rule is the project's actual differentiator — the architecture is table stakes.

Companion documents: the Faultline specification (architecture, safety, failure scenarios, interview prep) and the original three-project selection brief.